High Level Design

On

Entertainment Content Generator

Course Name: Generative AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Maitry Banduke	EN22CS301571
2.	Mohd Quasim	EN22CS301603
3.	Mrunali Kamerikar	EN22CS301618
4.	Muskan Asija	EN22CS301623

Group Name: G3 D6

Project Number: GAI -14

Industry Mentor Name:

University Mentor Name: Prof. Ajaj Khan

Academic Year: 2025-2026

Table of Contents

1. Introduction.

1.1 Scope of the document.

1.2 Intended Audience

1.3 System overview.

2. System Design.

2.1 Application Design

2.2 Process Flow.

2.3 Information Flow.

2.4 Components Design

2.5 Key Design Considerations

2.6 API Catalogue.

3. Data Design.

3.1 Data Model

3.2 Data Access Mechanism

3.3 Data Retention Policies

3.4 Data Migration

4.Interfaces

5.State and Session Management

6.Caching

7.Non-Functional Requirements

7.1 Security Aspects

7.2 Performance Aspects

8. References

1. Introduction

1.1 Scope of the Document

This High-Level Design (HLD) document defines the architectural blueprint, data structures, application flows, and deployment models for the **Entertainment Content Generator – an AI-powered system for content generation**.

The scope of this document covers the end-to-end lifecycle of the system, bridging the client-side presentation layer with the complex backend workflows required for Generative AI integration.

Specifically, this document outlines:

• The architecture separating the React-based frontend from the Python-based AI pipeline

• The internal mechanics of the Retrieval-Augmented Generation (RAG) system utilizing a FAISS vector index

• Data modeling strategies for handling unstructured vector embeddings and generated content

• Strict API contracts to govern the interaction between independent system modules

1.2 Intended Audience

This document is prepared as a technical guide for the following stakeholders:

• Software Engineers: To implement backend routing, APIs, and controllers
• AI/ML Developers: To build embedding pipelines and prompt engineering logic
• System Evaluators: To assess scalability and design decisions

1.3 System Overview

The Entertainment Content Generator is a **Generative AI-based system** designed to assist users in creating structured entertainment content such as scripts, dialogues, and story ideas. The system enhances the capabilities of standard Large Language Models (LLMs) by integrating a **Retrieval-Augmented Generation (RAG)** pipeline.

Unlike standalone LLM systems, which may produce generic or inconsistent outputs, this system improves accuracy and contextual relevance by incorporating external knowledge retrieved through a vector database.

When a user submits a prompt:

- The input is converted into vector embeddings using an embedding model
- Relevant contextual data is retrieved from the FAISS vector index using semantic similarity search
- A structured meta-prompt is constructed by combining user input and retrieved context
- The enriched prompt is sent to the LLM (Google Gemini API)
- The system generates structured and high-quality content

This approach ensures:

- Improved contextual relevance
- Consistent formatting
- Reduced hallucination in outputs
- Higher quality content generation

2. System Design

2.1 Application Design

The system follows a **modular and scalable architecture** consisting of three logical layers:

1. Client Layer (Frontend)

Implemented using React as a Single Page Application (SPA), this layer is responsible for user interaction and experience.

Responsibilities:

- Capturing structured user inputs (idea, genre, tone)
- Managing UI state and interaction flow
- Displaying generated outputs in formatted structure
- Providing responsive and real-time feedback

2. Application Layer (Backend)

Developed using Python (FastAPI/Flask), this layer acts as the central orchestrator of the system.

Responsibilities:

- Handling API routing and request validation
- Managing communication between modules
- Executing the RAG pipeline
- Integrating with external LLM services
- Formatting and returning responses

3. AI & Data Layer

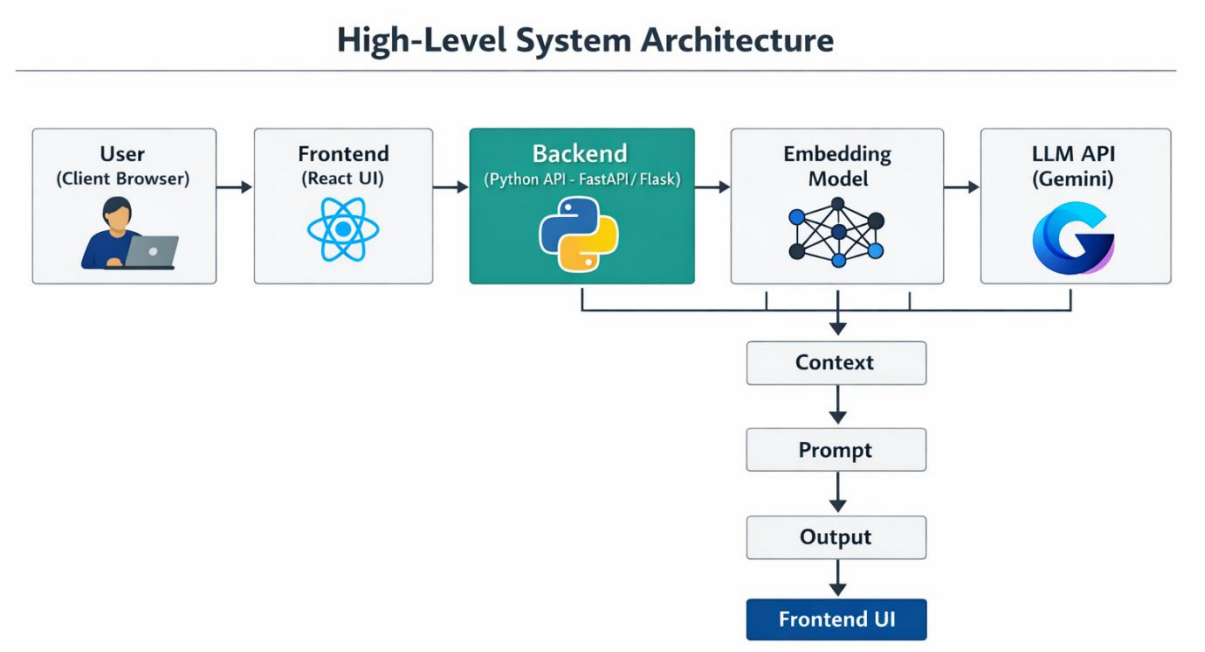
This layer handles embedding generation, vector storage, and LLM interaction.

Components:

- Embedding Model (e.g., MiniLM)
- FAISS Vector Index for similarity search
- Google Gemini API for content generation

Key Role:

- Enables semantic understanding of input
- Retrieves relevant contextual data
- Improves response accuracy through RAG



2.2 Process Flow

The script generation process follows a structured pipeline:

Step 1: Input Capture & Validation

The user submits input including idea, genre, and tone. Backend validates input using schema validation.

Step 2: Vectorization

Input text is converted into embeddings using an embedding model.

Step 3: Semantic Search

The embedding is queried against the FAISS vector index to retrieve relevant context.

Step 4: Prompt Construction

User input and retrieved context are combined into a structured prompt.

Step 5: LLM Execution

Prompt is sent to Google Gemini API.

Step 6: Post-Processing

Response is formatted and structured.

Step 7: Response Delivery
Final output is returned to the client.

2.3 Information Flow

Data moves in a structured pipeline ensuring predictable processing:

• Client sends request via HTTP

• Backend processes request

• Embedding model generates vectors

• FAISS retrieves relevant context

• LLM generates output

• Response returned to client

2.4 Components Design

The backend system is divided into modular components to ensure high cohesion and low coupling.

1. API Router Module

Responsibilities include:

• Handling incoming HTTP requests

• Managing routing of endpoints

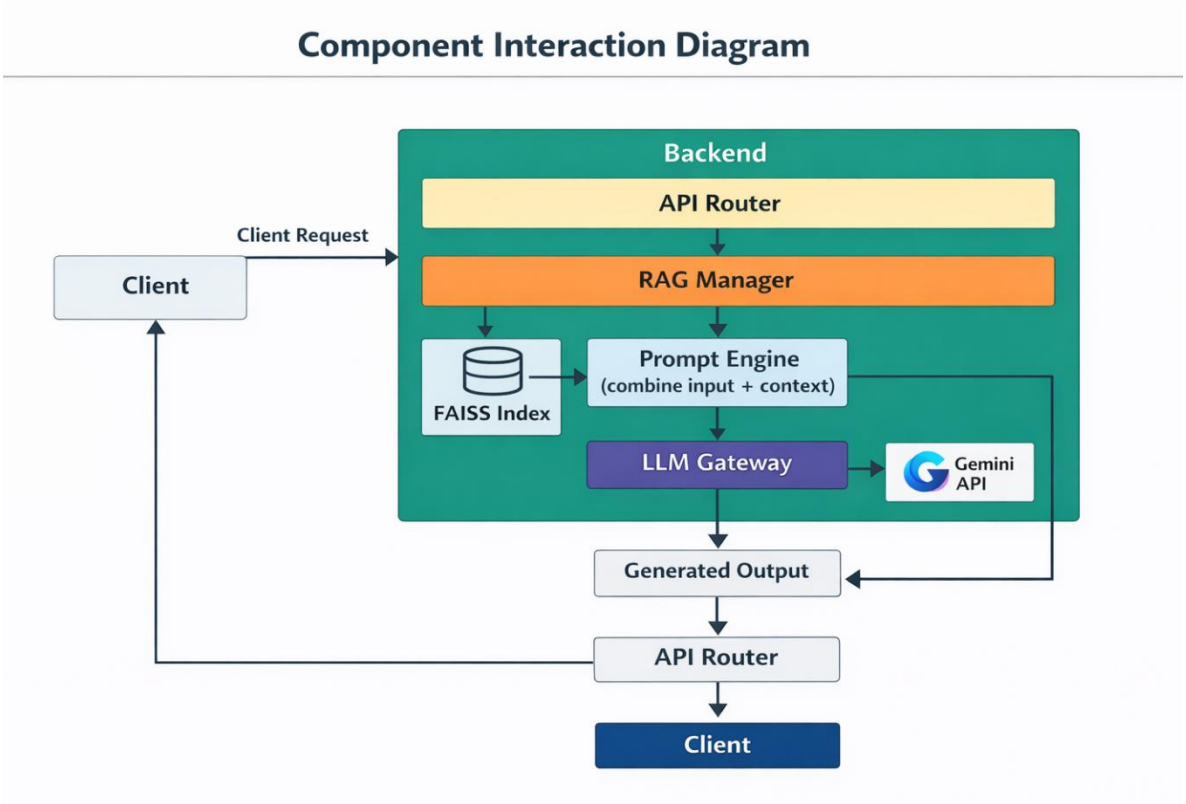
• Applying request validation

• Managing CORS policies

2. RAG Manager Module

Handles Retrieval-Augmented Generation logic:

• Managing interaction with the FAISS vector index
• Performing similarity search
• Retrieving top relevant documents



3. Prompt Engine Module

Responsibilities:

• Inject retrieved context into prompt
• Maintain formatting constraints
• Ensure LLM follows defined structure

4. LLM Gateway Module

Responsibilities:

• Communicates with Google Gemini API
• Handles request retries
• Manages response formatting
• Tracks API usage

5. FAISS Vector Index

Features:

• High-speed retrieval
• In-memory indexing
• Scalable for large datasets

2.5 Key Design Considerations

The system design is guided by the following key considerations:

1. Stateless Architecture

The backend is designed to be stateless, ensuring that each request is processed independently. This improves scalability and simplifies system design.

2. Token Optimization

To reduce computational cost and improve efficiency:

- Only relevant context is retrieved from the vector database
- Prompt size is controlled to avoid excessive token usage
- Redundant information is filtered

3. Efficient Processing

The system optimizes performance by:

- Using FAISS for fast vector similarity search
- Minimizing unnecessary API calls
- Structuring prompt generation for faster execution

4. Accuracy and Context Awareness

The use of RAG ensures:

- Better contextual understanding
- Reduced hallucination
- Improved quality of generated content

5. Scalability

The system can scale horizontally by:

- Deploying multiple backend instances
- Using stateless processing
- Handling independent API requests

6. Maintainability

- Modular design enables easy updates
- Separation of concerns improves debugging
- Components can be modified independently

2.6 API Catalogue

1. Script Generation Endpoint

Path: `/api/v1/generate/script`
Method: POST

Request Payload:

```
{
  "base_idea": "string",
  "parameters": {
    "genre": "string",
    "tone": "string",
    "character_focus": ["string"]
  }
}
```

Response Payload:

```
{
  "status": "success",
  "metadata": {
    "generation_time_ms": "approximate"
  },
  "data": {
```

```
    "content": "Generated script output...",
    "format": "markdown"
  }}
```

2. Vector Index Update Endpoint (Internal)

Path: /api/v1/admin/index/sync
Method: POST

Description:
Rebuilds FAISS index when new data is added.

3. Data Design

3.1 Data Model

The system operates on **unstructured textual data and vector embeddings**, rather than traditional relational or document-based databases.

Key Characteristics:

- Data is processed dynamically in real-time
- No persistent storage of user inputs
- Outputs are generated on-demand
- Embeddings are stored in a FAISS vector index

This design reduces storage overhead and enhances system performance while maintaining privacy.

3.2 Data Access Mechanism

• Index is loaded into memory at runtime
• Enables fast similarity search
• Uses efficient distance computation

3.3 Data Retention

• User inputs are not permanently stored
• Generated outputs are temporary
• Data exists only during processing lifecycle

3.4 Data Migration

Steps:

• Reprocess text corpus

• Generate new embeddings

• Rebuild FAISS index

• Replace old index

4. Interfaces

4.1 User Interface (UI)

Frontend developed using React.

Key Features:

• Fully decoupled from backend
• Communicates via REST APIs
• Responsive UI

Core Components:

• Workspace Dashboard

• Generation Panel

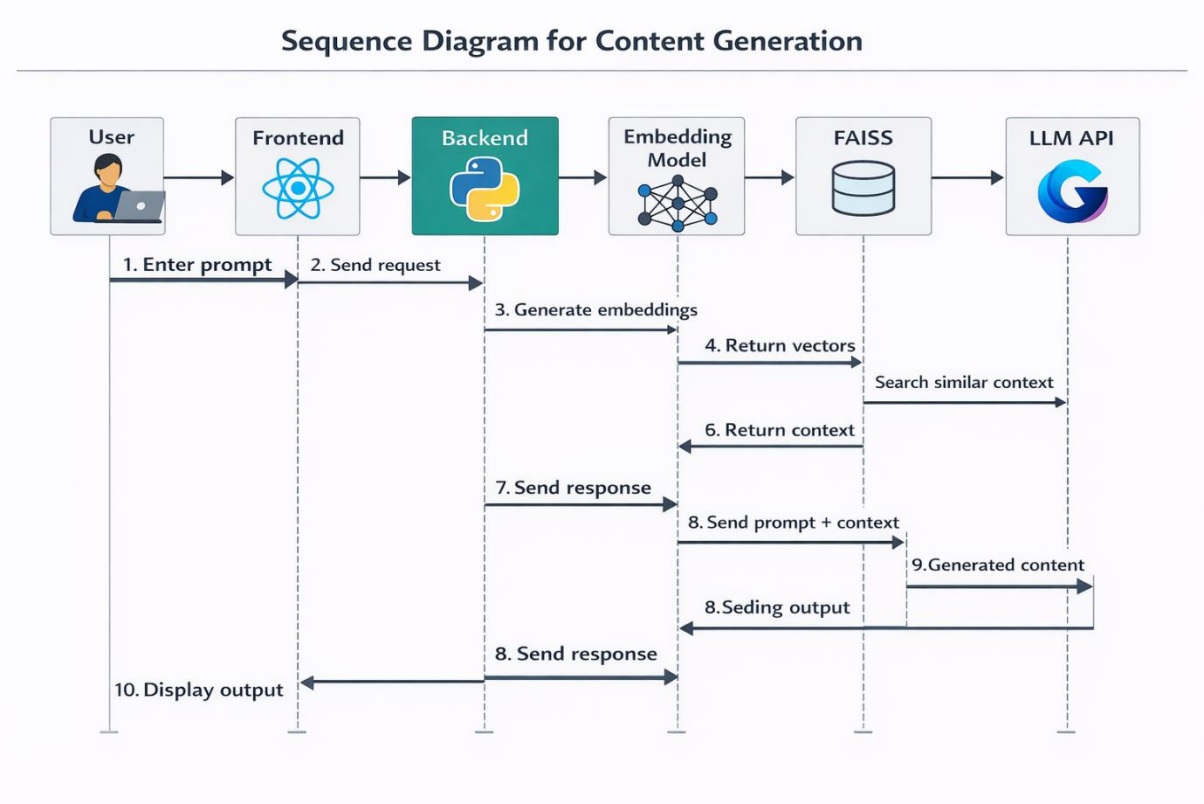
• Output Renderer

4.2 API Interfaces

• Uses HTTP methods
• JSON data format
• Versioned endpoints

4.3 External Services

• Google Gemini API (LLM)
• Local Embedding Model



5. State and Session Management

• Stateless backend
• No session storage
• Each request independent

6. Caching

Caching is implemented using lightweight in-memory techniques to improve performance and reduce redundant computations.

Key Features:

- Frequently used prompts and outputs are temporarily stored

- Reduces repeated LLM API calls
- Improves response time
- Cache is cleared automatically when the system resets

This approach ensures performance optimization without introducing complex caching infrastructure.

7. Non-Functional Requirements

7.1 Security

• HTTPS communication
• API keys via environment variables
• Input validation

7.2 Performance Aspects

The system is optimized to deliver fast and efficient responses while maintaining high-quality output generation.

Performance Strategies:

- Efficient FAISS-based similarity search for fast context retrieval
- Controlled prompt size to reduce LLM processing time
- Reduction of redundant computations
- Optimized API communication

Scalability:

- Stateless backend enables horizontal scaling
- Independent request handling supports multiple users
- Efficient memory usage through vector indexing

Reliability:

- Minimal dependency on persistent storage
- Modular design improves fault tolerance
- System can operate with partial component availability

8. References

1. Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. https://arxiv.org/abs/2005.11401
2. Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. IEEE Transactions on Big Data. (FAISS)
3. FastAPI Documentation. https://fastapi.tiangolo.com
4. React Documentation. https://react.dev
5. Google Gemini API Documentation. https://ai.google.dev
6. Sentence Transformers (MiniLM Embedding Model). https://www.sbert.net
7. REST API Design Best Practices. https://restfulapi.net